

Aug 25 - Aug 29

Format:

What you worked on

Key results/outcomes

Your plan for the next two weeks

Duy

What I worked on

- **The DeepSeek-V4-Flash out-of-memory crash.** It forces our deployment to process prompts in small pieces, which is slow. I investigated whether the setting could be raised and its cost, checked existing fixes, and ran four experiments on 4×H100.
- **Started GLM-5.2 AWQ.** This is our first end-to-end quantization on a model family other than MiniMax-M3.

Key results — DeepSeek-V4-Flash

- **Cause found.** Sparse attention builds a buffer sized *chunk × conversation length*, and the engine never budgets for it. That is **8 GB per GPU** at the 1M token limit, and it took production down on 17 Aug. This is a gap in one code path rather than a model flaw because an older version of the same codebase already has the missing check, making the fix low risk to adopt.
- **The upstream fix works and is free.** Without it, the server dies at an 800k-token prompt. With it, the server serves **1M tokens at 0.94–0.99×** of previous speed and holds at **95% KV cache occupancy**, where the incident occurred.
- **Raising the chunk size is not free.** 2048 to 4096 costs no KV cache; 8192 costs about **8%**. With prefill CUDA graphs enabled, the fix stops helping and 8192 would cost roughly **a third** of the cache, so that combination should be avoided.

Key results — GLM-5.2 AWQ

- **The blocker is slow weight loading, and it is storage.** The model is **1,485 GB** and shared storage reads at **~31 MB/s**: 86 minutes to walk layers we are not quantizing versus about 4 minutes to compress one. Holding 900 GB in RAM helped, but the real limit was the container's shared-memory size.
- **Now:** the 8-GPU AWQ run is ~54% through loading after 45 minutes. Several earlier AWQ and GPTQ runs failed, and I am working through them.

Plan for next week

- **GLM-5.2 AWQ to a finished model**, followed by the paired quality check against BF16. This item has the only deadline I do not control, so it comes first.
- **Recommend adopting the DeepSeek-V4-Flash fix**. Raising the chunk size is a separate call and now has numbers behind it.
- **Finish the serving measurements** that the memory work pushed aside.

## Alex

### What I worked on

The focus this week moved to aligning with the production side: identifying what actually gates deployment of the deliverable — the drafter model — and then improving the training recipe and data against their feedback and the problems it exposed.

**Reading the production-side signal.** The WEKA results reported by the deployment side are a false negative. The trace reproduces real request shapes but not real request content, so accept length measured on it says nothing about the drafter. What the report does tell us is what production actually cares about: long-context performance and quantization alignment.

**Scoping.** Two constraints shaped the rest of the week. First, current resources cannot support training another drafter at a different precision. Second, we are likely to refocus on GLM-5.3 once it is released. Rather than open the quantization thread, I worked on long-context improvement for GLM-5.2 with the existing DSpark drafter. There is real headroom there — the current drafter uses full attention and was trained on short contexts only.

**Direction 1: swap full attention for SWA at inference time.** Attention is typically localized over the context, so restricting it to a window should discard little that matters. speed-bench's throughput split contains samples from 1k to 32k, so I used it to test context-length generalization at fixed concurrency (c1).

**Direction 2: long-context fine-tuning of the trained DSpark, adapting to SWA during training.** The DFlash paper reports that a light fine-tune on 1.6k LongAlign samples improves long-context behaviour dramatically — accept length even rises as context grows. The paper does not say how those 1.6k samples were selected, so I bucketed LongAlign by length and drew 1.6k uniformly across buckets.

**Failure-mode analysis.** I also looked at patterns in the samples where the drafter performs worst, as input to curating training data more deliberately.

### Key results / outcomes

- Switching full attention to SWA almost entirely removes the degradation that comes with longer context. At ~8k, full attention has effectively stopped speculating (AL 1.22), while SWA-4096 holds accept length at 3.90 and cuts ITL from 20.2 ms to 6.6 ms. The cost is a small regression at ~1k, in both accept length and ITL.

| ISL        | tier        | full attention AL | swa4096 AL | change | full ITL (ms) | swa4096 ITL (ms) |
|------------|-------------|-------------------|------------|--------|---------------|------------------|
| ~1k (982)  | low_entropy | 3.80              | 3.65       | -4%    | 6.05          | 7.01             |
| ~8k (7929) | low_entropy | 1.22              | 3.90       | +220%  | 20.20         | 6.59             |

- The fine-tuning experiments are still running. (will be able to updated before the next Tuesday)

## Blockers and Plan for the next two weeks

- The bottleneck of resources from last week persists, and left three items unstarted: drafter algorithm improvements, DSpark training for DeepSeek-V4-Flash, and DFlash-2 training for DeepSeek-V4-Flash.
- Run the three blocked items in parallel as soon as allocation allows: drafter algorithm improvements, DSpark for V4-Flash, and DFlash-2 for V4-Flash.
- Finish the long-context fine-tuning runs, and if the SWA-aware fine-tune holds up, fold SWA into the training recipe.
- Continue the training-data work — failure-pattern analysis and curation. This is worth treating as a long-running thread rather than a one-off.

## ZHOU YU

### What I worked on

- Improved the Agentic evaluation pipeline:** Integrated the Bitdeer AI Cloud API and completed the SWE-bench Pro workflow from patch generation through official evaluation and result aggregation.
- Upgraded benchmark implementations:** Migrated BFCL to official V4, including memory, web search, and weighted scoring; aligned MRCR token bins and sample reporting with the official methodology.
- Implemented distributed evaluation:** Added Terminal Bench scheduling across four workers and eight execution slots, supporting five repetitions, trial retries, recovery, and result merging. Built a live monitoring dashboard.

### Key results/outcomes

- **DeepSeek-V4-Flash results:** Tool Schema achieved **26/26 (100%)**. Legacy BFCL 6-B achieved **3,508/4,981 (70.43%)**; this is not a BFCL V4 score.
- **Pipeline validation:** Verified the complete SWE-bench Pro workflow. BFCL V4 passed its single-case smoke test (**1/1**); a full V4 result is not yet available.
- **MRCR:** A long-context single-case test scored approximately **0.9994**. Both full evaluation attempts failed due to API 503 errors, so no formal score was produced.
- **Terminal Bench 2.1:** The four-worker smoke test passed **8/9** tasks. Started the full GLM-5.2 evaluation with 256K context and **89 tasks × 5 repetitions**. As of **August 28, 14:17 SGT**, **83/445 trials** were completed, with **73.49% accuracy (61/83)** on completed trials. These are provisional results.

### Plan for the next week

- **Integrate the Simple Agentic Bench Version( [quant-bench-list-simple \(1\).html](#) )**
  - Review the dataset format, execution protocol, and scoring method of the bench in attached document.
  - Add an independent driver, result adapter, and tests following the existing modular structure.
  - Integrate it with the common Orchestrator and complete a small-scale end-to-end validation.
- **Integrate with GitLab Runner**
  - Package the benchmark workflow as GitLab CI/CD jobs.
  - Support parameterized selection of models, benchmarks, dataset subsets, and concurrency settings.
  - Provide simple operations for launching tasks, checking status, collecting logs, and archiving results.